Member 1 — Firmware and System Integration

Member placeholder: [MEMBER 1 NAME]

Proposed viva responsibility: firmware architecture, sensor integration, acquisition scheduling, validity-state propagation, telemetry serialization, OLED, SOS/fall paths and build integration.

Authorship disclaimer: this document records the capabilities the repository supports and a proposed responsibility allocation for the HOD discussion. It does **not** prove that [MEMBER 1 NAME] wrote every line in src/main.cpp. The team must confirm individual history. Until then, say **"the team implemented this area"** for source-backed claims.

A. Responsibility scope

A.1 One-sentence scope

For this discussion, [MEMBER 1 NAME] is responsible for explaining how the ESP32-S3 firmware turns three installed sensors, one OLED and a pushbutton into a coherent, null-safe telemetry node — and for being honest about what has not been verified on hardware.

A.2 Included in this responsibility

- The firmware entry points, setup() and loop().
- I2C bus initialization and checked register helpers.
- Initialization and scheduling for MAX30102, MPU-6050/MPU-6500 and TMP117.
- Sensor health/status propagation (valid, warming_up, low_quality, stale, sensor_error, not_available).
- The one-line JSON telemetry contract at schema 2.
- Actual-sensor versus clinical-wrist field separation at the firmware boundary.
- SOS button behavior and the two-stage fall-candidate path.
- Hot-plug removal/recovery checks for installed devices.
- OLED rendering and its acknowledged compact-text limitation.

- Build configuration in platformio.ini and the custom N16R8 board manifest.
- Explaining how these outputs are consumed by the bridge, dashboard, store and tests.

A.3 Outside this member's primary scope — but still required knowledge

- Deep TMP117 electrical/protocol validation and reference-thermometer testing: Member 2.
- Python serial parser, HTTP/SSE backend and schema enforcement: Member 3.
- SQLite schema/history queries and dashboard DOM code: Member 4.
- Temperature safety policy, baseline statistics, deterministic alert rules and audit evidence: Member 5.

The boundaries are interfaces, not excuses. Member 1 must still know the exact temperature semantics and the bridge's rejection rules.

A.4 Files that define the scope

File	Why it matters
src/main.cpp	1,788-line firmware: I2C, acquisition, status, JSON, OLED, alerts
src/tmp117_protocol.h	TMP117 register definitions and pure conversion/range helpers used by firmware and diagnostic
src/sensor_math.h	MPU scale conversions and MAX optical-quality guards
platformio.ini	Main build, optional modules compiled out, monitor/upload settings
boards/esp32-s3-devkitc-1-n16r8.json	16 MB QIO flash, 8 MB OPI PSRAM board definition
diagnostics/tmp117_raw/	A separate build used when the TMP117 needs raw-register investigation
tests/test_tmp117_vectors.py	Host evidence for the conversion formulas used by the firmware header
telemetry_bridge.py	The strict consumer of the firmware's JSON; integration contract

B. Achievements attributable to this area

These are **area achievements**, not proof of personal authorship.

Achievement	What the repository actually contains	Evidence
One integrated firmware image	OLED, MAX30102, MPU and TMP117 initialization in one setup(); periodic acquisition in one loop()	[SOURCE] src/main.cpp
Documented one-line contract	JSON schema 2, [JSON] prefix, 115200 baud, rpm-2.1.0	[SOURCE] outputTelemetry(); [TEST] bridge parser tests
Fail-closed value policy	Stale/missing/error values serialize as null; refreshSensorStatuses() runs before urgent output	[SOURCE] refreshSensorStatuses(), printNullable(); host tests exercise null handling
Actual/clinical temperature separation	tmp117_sensor_temperature_* fields separate from wrist_skin_temperature_*	[SOURCE]; [TEST] Python and Node temperature tests
Conservative clinical wrist output	Always wrist_contact_status: "unverified" and null wrist value/raw/timestamp	[SOURCE]; [TEST] test_temperature_safety.py
SOS and fall candidate paths	Active-low button with debounce/cancel; two-stage free-fall/impact heuristic	[SOURCE]; physical behavior [UNVERIFIED HW]
Hot-plug handling	3 s retry window; TMP117 and MAX removal checks; recovery init	[SOURCE]; physical removal/reconnect [UNVERIFIED HW]
Reproducible builds	Main and diagnostic builds succeed	[BUILD] fresh on 2026-10-29
Honest optional-module handling	SHT40/MLX90614 compiled out and never counted	[SOURCE]; [TEST] bridge rejects claimed presence

B.1 What this area does not establish

- That the sensors are electrically connected correctly on the assembled board.

- That the TMP117 address 0x49 (prior observation) is the actual fitted address now.
- That MAX30102 HR/SpO2 are accurate against a reference.
- That MPU axes, signs, ranges or fall thresholds behave correctly in the enclosure.
- That the OLED is legible in the physical build.
- That the device is safe for a patient or suitable as an emergency device.

C. Detailed code walkthrough

C.1 Boot sequence — setup()

setup() at src/main.cpp:1615 performs the following in order:

1. Serial.begin(115200) and a short startup delay.
2. Prints the board model, flash size and PSRAM size; prints the temperature policy.
3. Wire.begin(I2C_SDA_PIN, I2C_SCL_PIN, I2C_PRIMARY_HZ) with GPIO 8/9 at 50 kHz; Wire.setTimeOut(25).
4. Wire1.begin(MPU_SDA_PIN, MPU_SCL_PIN, I2C_IMU_HZ) with GPIO 17/18 at 400 kHz; timeout 25 ms.
5. Configures GPIO 5 as INPUT_PULLUP for the active-low SOS button.
6. Scans the primary I2C bus (scanI2CBus()), then initializes the OLED at 0x3C, MPU, MAX30102 and TMP117.
7. Prints the installed measurement count using activeVitalSensorCount() over the fixed denominator VITAL_SENSOR_MAX = 3.
8. Prints that the OLED is a display and that SHT40/MLX90614 are absent.
9. Initializes the report/fast-tick/retry timers and calls outputTelemetry() once.

Why the order matters: the bus must be configured before any sensor initialization; the OLED is useful for boot feedback but is not fatal; TMP117 initialization waits 600 ms after configuration so the first 500 ms conversion can complete; the first telemetry packet is emitted only after all initialization attempts have produced a status.

C.2 Main scheduling — loop()

loop() at src/main.cpp:1712 is cooperative rather than an RTOS task design:

Work	**Cadence / trigger**	**Function**
Clear SOS/fall timers	Every loop when expired	inline checks in loop()
SOS button edge detection	Every loop; 300 ms debounce	digitalRead() + state
MPU acquisition	Every 20 ms	updateMpu(now)
MAX30102 sampling	Every loop, gated internally by 150 ms gap	sampleOptical(now)
TMP117 acquisition	Every 2 s when present	updateTmp117(now)
Hot-plug/status refresh and report	Every 1 s	refreshSensorStatuses(), updateSensorsHotplug(), outputTelemetry(), updateOLED()

A simplified representation:

```
void loop() {
  uint32_t now = millis();

  if (sosActive && now >= sosClearTime) sosActive = false;
  if (fallActive && now >= fallClearTime) fallActive = false;

  // active-low button: 300 ms debounce, first press starts SOS,
  // a second press cancels; start writes immediate telemetry.
  // ...

  if (now - lastFastTick >= MPU_READ_INTERVAL_MS) {
    lastFastTick = now;
    updateMpu(now);
  }
```

```
  sampleOptical(now);
  updateTmp117(now);

  if (now - lastReportTime >= 1000) {
    lastReportTime = now;
    refreshSensorStatuses(now);
    updateSensorsHotplug(now);
    // prompt removal checks for OLED/MAX/TMP117
    outputTelemetry();
    updateOLED();
  }
}
```

The code is not a hard real-time scheduler. A long I2C timeout can delay the loop. The 25 ms Wire timeout is a bounded mitigation, not a guarantee of deterministic patient monitoring.

C.3 Checked I2C helpers

The firmware does not assume that a read succeeded. The helper stack in src/main.cpp includes:

- enum class I2CResult { OK, ADDRESS_NACK, DATA_NACK, OTHER_ERROR, SHORT_READ };
- i2cResultFromEndTransmission();
- i2cResultName();
- readRegisterBytes(), readRegisterWord(), readRegisterByte();
- writeRegisterWord(), writeRegisterByte();
- sensorResponds().

A simplified read path:

```
I2CResult readRegisterBytes(TwoWire& bus, uint8_t address,
                            uint8_t reg, uint8_t* destination, size_t length) {
  bus.beginTransmission(address);
  bus.write(reg);
  I2CResult result = i2cResultFromEndTransmission(bus.endTransmission(false));
  if (result != I2CResult::OK) return result;

  if (bus.requestFrom((int)address, (int)length) != length)
    return I2CResult::SHORT_READ;

  for (size_t i = 0; i < length; ++i) {
    if (!bus.available()) return I2CResult::SHORT_READ;
    destination[i] = (uint8_t)bus.read();
  }
  return I2CResult::OK;
}
```

The important principle is that an I2C failure cannot become a plausible-looking sensor value. This is also why the TMP117 diagnostic avoids the Adafruit TMP117 library and uses checked raw transactions.

C.4 Sensor acquisition and status propagation

Each sensor has a presence flag, a status string, an error string and one or more freshness timestamps.

TMP117

● `initTmp117()` scans the candidate addresses `{0x48, 0x49, 0x4A, 0x4B}`.

- It verifies `WHO_AM_I (0x0F) == 0x0117`.
- It writes configuration `0x01A0` and verifies the writable-mask readback.
- `updateTmp117()` calls `readTmp117()` every 2 s.
- `readTmp117()` checks EEPROM busy and `DATA_READY` before reading temperature, rejects `0x0000/0xFFFF`, enforces `-55..150 °C`, and requires a confirming sample after an abrupt change greater than 5 °C.
- `tmp117ValueValid()` requires presence, valid raw, valid status and a finite temperature.
- A removed device is detected promptly by an address probe in the 1 s report path and by `ADDRESS_NACK` during a read.

Member 2 owns the deeper protocol justification; Member 1 must be able to explain the interface and why the actual sensor reading is not the clinical wrist reading.

MPU-6050/MPU-6500

- `initMpu()` tries `0x68` then `0x69`; `initMpuRaw()` verifies `WHO_AM_I` values `0x68` or `0x70`.
- It resets power management, sets sample-rate divider, digital low-pass filter, gyro range ±250 dps and accel range ±8 g.
- `calibrateMpu()` attempts a stationary 80-sample startup bias estimate; this is **not** a full six-face calibration.
- `updateMpu()` reads 14 bytes, converts raw values with `sensor_math` (`4096 LSB/g`, `131 LSB/dps`), calculates magnitudes, orientation and activity, and updates steps/fall candidate logic.
- `refreshSensorStatuses()` marks MPU stale after `MPU_STALE_MS = 1500 ms`.

MAX30102

- `initMax30102()` checks the part ID and configures the sensor; failures set `maxPresent = false` and non-valid statuses.
- `sampleOptical()` collects FIFO samples, rejects DC values outside `50000..262143`, and after a 100-sample window calls `finishOpticalWindow()`.
- `finishOpticalWindow()` computes minimum/maximum/mean, perfusion index using `sensor_math::perfusionIndex()`, and applies signal-quality guards before calling the SparkFun algorithm.

- heartRateValueValid() and spo2ValueValid() require presence, valid status, finger detection and a positive/range-valid result.
- refreshSensorStatuses() marks optical data stale after OPTICAL_STALE_MS = 2000 ms.

C.5 SOS and fall paths

SOS (GPIO 5, active LOW):

- Internal pull-up; a press reads LOW.
- Edge detection uses a 300 ms debounce (now - lastButtonPress > 300).
- First press sets sosActive = true, starts a 15 s clear timer, prints an emergency line, and calls outputTelemetry(true) and updateOLED() immediately.
- A second press cancels it and emits an immediate packet.
- The 15 s timer auto-clears the latch.
- This is a local prototype alert. It does not dispatch an ambulance and is not an emergency-response system.

Fall candidate:

- currentMotionG < 0.6 g records freeFallTime.
- A free-fall record older than 1000 ms is cleared.
- An impact above 2.5 g within 60..600 ms of free-fall sets fallActive = true for 6 s, emits urgent telemetry, and updates the OLED.
- It is a two-stage heuristic, not a validated fall detector.

C.6 Telemetry serialization — outputTelemetry()

outputTelemetry(bool urgent) is the firmware's public contract builder. It:

1. Calls refreshSensorStatuses(now) first, even on urgent paths, so stale values are not published as valid.
2. Computes tmpValid, imuValid, hrValid, spo2Valid consistently with the status model.
3. Writes a single line beginning [JSON] { — no newline inside the object.
4. Emits top-level uptime_s, device_id, firmware_version, schema_version, board, risk, urgent.

5. Emits sensors with OLED/MAX/MPU/TMP117 booleans, sht40:false, mlx:false, active and active_max:3.

6. Emits sensor_health with error/calibration details.

7. Emits vitals with printNullable() so invalid values become null.

8. Always emits the actual TMP117 fields separately; if invalid, value/raw/timestamp are null.

9. Always emits clinical wrist contact unverified, wrist value/raw/timestamp null.

10. Always emits core null/not_available and fever not_available; the exact message is "Core temperature unavailable — confirmation required".

11. Emits alerts and status_summary, then a newline.

The actual temperature block is deliberately conservative:

```
Serial.print(F(",₩"tmp117_sensor_temperature_c₩":"));

printNullable(tmpValid ? tmp117TempC : NAN, 2);

Serial.print(F(",₩"tmp117_sensor_temperature_status₩":₩""));

printStatus(tmp117Status);

Serial.print(F("₩",₩"tmp117_sensor_temperature_raw₩":"));

if (tmpValid) Serial.print(tmp117Raw); else Serial.print(F("null"));

Serial.print(F(",₩"tmp117_sensor_temperature_timestamp_ms₩":"));

if (tmpValid) Serial.print(tmp117LastValidMs); else Serial.print(F("null"));


Serial.print(F(",₩"wrist_contact_status₩":₩"unverified₩""));

Serial.print(F(",₩"wrist_skin_temperature_c₩":null"));

Serial.print(F(",₩"wrist_skin_temperature_status₩":₩"unverified₩""));

Serial.print(F(",₩"wrist_skin_temperature_raw₩":null"));

Serial.print(F(",₩"wrist_skin_temperature_timestamp_ms₩":null"));
```

This is the firmware side of the rule: **actual sensor temperature is not promoted to a clinical wrist value.**

C.7 OLED

updateOLED() uses pages of compact text on a 128×64 SSD1306. It shows:

- SOS/fall emergency banner when active;
- current HR/SpO2 and finger state;
- actual TMP117 temperature with a compact TMP: prefix;
- wrist status with a compact Wrist: prefix;
- motion/orientation on an alternating page;
- compact core/fever wording.

The display is not a clinical labeling surface. The exact required labels and full fever message are implemented in the dashboard and JSON; the OLED is constrained by its font and size. Its physical readability is [UNVERIFIED HW].

C.8 Build integration

platformio.ini:

- targets esp32-s3-devkitc-1-n16r8;
- monitor 115200, upload 460800;
- monitor_dtr = 0 and monitor_rts = 0;
- -DRPM_ENABLE_OPTIONAL_SHT40=0;
- -DRPM_ENABLE_OPTIONAL_MLX90614=0;
- optional libraries are not required.

The custom boards/esp32-s3-devkitc-1-n16r8.json selects 16 MB QIO flash, 8 MB OPI PSRAM and the 16 MB partition table. Using the stock N8 definition would silently change flash/PSRAM assumptions.

D. Technical decisions and trade-offs

Decision	Reason	Trade-off / limitation
Two I2C buses	Keep the IMU at 400 kHz away from slower primary	More pins and wiring; both buses still need correct

Decision	Reason	Trade-off / limitation
	devices; reduce bus contention	pull-ups/ground
Checked raw Wire transactions	I2C errors cannot become valid-looking temperatures	More code than a library; needs hardware verification
One-second JSON cadence	Good balance for local dashboard/history	Not continuous waveform telemetry; 20 ms MPU data are reduced to latest state
Immediate urgent packet	SOS/fall should not wait for the 1 s report tick	Can produce extra packets; bridge must suppress exact duplicates
null for unavailable values	Prevents zero/stale substitution and false clinical display	Downstream code must handle null everywhere; UI cannot show a number
Fixed VITAL_SENSOR_MAX = 3	Installed inventory is MAX/MPU/TMP117; optional modules must not inflate the denominator	A future hardware revision must deliberately change this constant and schema assumptions
Firmware always wrist_contact_status: unverified	No validated contact detector exists	Clinical wrist value is unavailable even when TMP117 is working
TMP117 2 s read / 500 ms conversion	Sensor is configured for 8-sample averaging at 500 ms; 2 s reduces bus load	Slower than the 1 s telemetry cadence; the same accepted value may be republished
Hot-plug retry every 3 s	Recover a removed/reconnected device without reboot	Can add bus traffic; detection is not instantaneous
OLED compact labels	128×64 and built-in font constrain text	Cannot show the exact long clinical label/message; not a substitute for dashboard labels
Compile out SHT40/MLX	Avoid presenting absent hardware as available	Legacy code paths remain in source but are disabled; do not claim them as features

D.1 Failure philosophy

The software prefers a visible **"unavailable"** over a plausible number. This is not just UI

polish: it prevents an absent sensor, an unverified wrist contact or a stuck bus from becoming a clinical observation. The cost is that the system often shows fewer numbers, and the baseline cannot qualify while wrist temperature is missing.

E. Cross-module contributions and interfaces

E.1 Firmware → bridge contract

The bridge relies on these firmware guarantees:

- [JSON] line prefix and one-line object;
- schema 2;
- sensors.active equals the count of true installed sensors and active_max is 3;
- sht40/mlx are false;
- a valid value has a present sensor and required evidence;
- a non-valid value has null value/raw/timestamp where the schema requires it;
- actual TMP117 and clinical wrist are separate;
- core is null and fever is not_available;
- timestamps are non-negative integers when present.

If the firmware violates any of these, _validate_packet() rejects the whole packet. That is intentional: a broken packet should not partially update the live view.

E.2 Firmware → storage/dashboard

- The dashboard reads sensors, sensor_health, top-level alerts and vitals fields; activeSensorCount() counts only the three installed sensors.
- The store uses measurements() to retain only valid values; the actual TMP117 field is not written as the clinical wrist column.
- The exact dashboard labels are defined in assets/dashboard.js and assets/dashboard_core.js; the firmware's OLED uses compact text.

E.3 Firmware → safety/baseline

- temperature_safety.evaluate_temperature_safety() re-derives the actual/clinical semantics from the packet, rather than trusting a label.
- PersonalBaseline uses telemetry_store.measurements(), so unverified wrist values cannot feed a temperature baseline.
- The deterministic engine reports actual TMP117 as actual sensor telemetry and never as core.

E.4 Integration contribution to state clearly

Member 1 can say:

> "My integration responsibility was to ensure that the firmware's JSON is not merely a collection of readings, but a state contract: presence, status, raw evidence and nullability travel together so the backend can reject contradictions."

Do not claim authorship of the whole firmware unless the team confirms it.

F. Evidence classification for this area

Claim	Classification	Where to verify
Firmware initializes OLED, MPU, MAX30102, TMP117	[SOURCE]	setup(), init functions
Firmware emits schema 2 JSON with [JSON] prefix	[SOURCE]	outputTelemetry()
Firmware separates actual TMP117 from clinical wrist	[SOURCE] + [TEST]	outputTelemetry(), Python/Node tests
Stale statuses are refreshed before output	[SOURCE]	refreshSensorStatuses() called at top of outputTelemetry()
SOS/fall paths call urgent output	[SOURCE]	loop() + updateMpu()
Main firmware builds	[BUILD] fresh 2026-10-29	RAM 21,424 / 327,680; flash 326,001 / 6,553,600; SUCCESS
TMP117 raw diagnostic builds	[BUILD] fresh 2026-10-29	RAM 18,916 / 327,680; flash 282,377 / 6,553,600; SUCCESS
62 Python / 10 Node tests pass	[TEST] fresh 2026-10-29	python -m pytest -q, node --test

Claim	Classification	Where to verify
		tests/dashboard_core.test.js
Physical SOS/fall/OLED behavior	[UNVERIFIED HW]	Requires assembled-device test
Sensor accuracy	[UNVERIFIED HW]	No reference measurements
TMP117 -0.01 °C cause	[RECORDED HW OBS] hypothesis only	Raw diagnostic log still required

F.1 Prior TMP117 observations — do not overstate

The audit records prior observations of:

- raw 0x0ED3 → 29.6484375 °C;
- raw 0xFFFF → -0.0078125 °C, rounded to -0.01 °C on a two-decimal display;
- diagnostic raw 0x0DA7 → 27.30469 °C;
- 60 reads with 0 failures and 0 stuck patterns;
- a prior address observation at 0x49.

These were not reproduced in the 2026-10-29 documentation pass. The correct evidence label is **prior hardware observation only; hardware verification required.** They do not prove the current board is working or accurate.

G. Debugging: realistic scenarios and correct reasoning

No claim is made that these exact logs were produced in this pass. They are the debugging paths supported by the source and audit records.

G.1 “The display shows about -0.01 °C”

Symptom: a two-decimal TMP117 display can show -0.01 °C.

Hypothesis: raw 0xFFFF is a stuck-bus/all-ones pattern; signed conversion gives -0.0078125 °C, which rounds to -0.01 °C.

Correct response:

1. Do not add a software offset.
2. Run diagnostics/tmp117_raw on the assembled board.

3. Capture at least five minutes, including the event.

4. Record address, `WHO_AM_I`, configuration readback, offset, raw bytes, converted value, `DATA_READY`, failures, all-ones/all-zeros counts.

5. Confirm/refute whether `0xFFFF` is the cause before claiming it.

Current status: hypothesis only; **HARDWARE VERIFICATION REQUIRED**.

G.2 "TMP117 not found"

Possible causes the code distinguishes or exposes:

- no device ACKs at `0x48–0x4B`;
- `WHO_AM_I` mismatch;
- configuration readback mismatch;
- bus/power/ground/wiring problem.

The boot output prints a found address and identity; the status/error fields expose failure. The correct next step is the raw diagnostic and wiring inspection, not changing the range to accept a wrong value.

G.3 "The dashboard shows stale numbers"

The firmware calls `refreshSensorStatuses()` before every output, including urgent paths. Staleness thresholds:

- TMP117: 6000 ms;
- MPU: 1500 ms;
- optical: 2000 ms;
- sensor retry: 3000 ms.

The bridge additionally expires the live view after 5 s without a fresh packet. If a value is stale, both layers should show unavailable, not the old number. Verify timestamps and statuses rather than assuming the chart is wrong.

G.4 "Sensor removed/reconnected"

The 1 s loop path probes the OLED, MAX30102 and TMP117. A removed TMP117 becomes

not_available, its value/raw are cleared and the offset/presence flags are invalidated; updateSensorsHotplug() retries every 3 s. This is implemented in source; physical removal/reconnect remains [UNVERIFIED HW].

G.5 "JSON rejected by the bridge"

The bridge rejects the entire packet for conditions such as:

- invalid schema or missing dictionaries;
- duplicate JSON keys;
- NaN/Infinity;
- wrong active/active_max;
- absent sensor with valid status;
- non-null value with a non-valid status;
- TMP117 raw/conversion mismatch or stuck pattern;
- wrist value without verified contact and matching actual-sensor evidence;
- non-null core or missing fever message.

Debugging rule: inspect the packet against the contract; do not weaken the bridge to accept a malformed packet. If the firmware emits a bad packet, fix the producer or document the schema change.

G.6 "OLED text is truncated"

The display is 128×64 with a fixed font. The source uses compact TMP:/Wrist: strings and two-line core wording. The exact clinical labels are in the dashboard/JSON. The physical readability and exact rendering are not verified. Do not claim the OLED displays the full required label.

H. Testing and verification

H.1 Fresh checks in this documentation pass

Command	Result
python -m pytest -q	62 passed

Command	Result
node --test tests/dashboard_core.test.js	10 passed
Ruff / Python compilation / JavaScript syntax	passed
pio run -e esp32-s3-devkitc-1	SUCCESS; RAM 21,424/327,680; flash 326,001/6,553,600
pio run -d diagnostics/tmp117_raw	SUCCESS; RAM 18,916/327,680; flash 282,377/6,553,600

H.2 Firmware-relevant tests

- tests/test_tmp117_vectors.py checks datasheet conversion vectors, including 0x0ED3 → 29.6484375, 0xFFFF → -0.0078125, 0x8000 → -256.0, and the all-ones anomaly.
- src/tmp117_protocol.h has compile-time assertions for conversion and range helpers.
- src/sensor_math.h has compile-time checks for MPU scale constants and optical guard relationships.
- Bridge/backend tests exercise schema, status/value consistency and status propagation.
- Python temperature tests exercise actual-versus-unverified and verified-contact paths.

H.3 What the tests do not prove

- The sensors exist or respond on the actual board.
- TMP117 accuracy or the absence of self-heating.
- MAX30102 HR/SpO2 accuracy under motion, ambient light or poor perfusion.
- MPU sign/axis/range correctness or fall sensitivity.
- SOS/fall latency and OLED visibility.
- That the JSON seen on a real serial line has no electrical corruption.

H.4 Minimum hardware validation sequence for this area

1. Inspect power/ground and 3.3 V compatibility.
2. Run the TMP117 raw diagnostic and capture five minutes.
3. Upload main firmware; observe boot scan and 3/3 measurement sensors online when all

are present.

4. Remove/reconnect each installed sensor and verify status/null/recovery.
5. Press SOS and test cancel/auto-clear; safely simulate a fall.
6. Compare actual TMP117 with a reference; do not call the package temperature a wrist or core measurement.
7. Confirm OLED legibility and compact-text behavior.
8. Record results in a hardware log, not by weakening source checks.

I. HOD preparation

I.1 60–90 second explanation

Use approximately 170–190 words; practice to 60–90 seconds.

> "My allocated area is the ESP32-S3 firmware and system integration. The firmware initializes an OLED, a MAX30102 optical sensor, an MPU-6050 or MPU-6500 on a separate I2C bus, a TMP117 and an SOS button. The main loop reads the IMU every 20 milliseconds, samples the optical sensor, reads the TMP117 every two seconds, and once per second refreshes every sensor status before emitting one-line JSON at 115200 baud. SOS and fall candidates emit immediate packets rather than waiting for the next second. The key design decision is that the firmware never fabricates a measurement: invalid, stale or absent values become JSON null with an explicit status, and urgent output still refreshes freshness first. It also keeps the actual TMP117 sensor reading separate from the clinical wrist-skin field. The wrist field is always null and unverified because no validated contact detector exists. The firmware builds successfully and the host tests pass, but the physical sensors, OLED, SOS and fall behavior have not been validated in this audit. So I can defend the integration and data contract, not claim clinical accuracy."

I.2 Technical questions and answers

1. **What does `setup()` do before the first telemetry packet?** It starts serial at 115200, configures both I2C buses, sets the SOS pin, scans the bus, initializes OLED/MPU/MAX30102/TMP117, prints the 3/3 inventory, initializes timers and calls `outputTelemetry()`.
2. **Why are there two I2C buses?** The primary bus carries the OLED, MAX30102 and TMP117 at 50 kHz; the dedicated IMU bus carries the MPU at 400 kHz. This isolates the faster

IMU traffic and reduces contention.

3. **What is the JSON report cadence and urgent behavior?** The normal path reports every 1000 ms. SOS/fall call `outputTelemetry(true)` immediately; that function refreshes statuses first so stale values cannot be published as valid.

4. **How does the firmware represent a missing value?** `printNullable()` emits `null` when the value is not finite or not valid; status fields explain why. It never substitutes zero or the previous value.

5. **What makes a TMP117 value valid in the firmware?** `tmp117ValueValid()` requires `tmp117Present`, `tmp117RawValid`, status `valid` and a finite `tmp117TempC`; the read path additionally enforces identity, configuration, `DATA_READY`, non-stuck raw, range and abrupt-change confirmation.

6. **How does the firmware distinguish actual sensor temperature from clinical wrist temperature?** It writes `tmp117_sensor_temperature_c` from the actual TMP117 read and always writes `wrist_contact_status: "unverified"` with null wrist value/raw/timestamp. It has no code path that copies the actual value into the wrist field.

7. **Why does the firmware not estimate core temperature?** There is no validated core measurement or transformation. `estimated_core_temperature_c` is always null and its status is `not_available`; fever is `not_available` with the exact message.

8. **How does the SOS path work?** GPIO 5 is active LOW with pull-up. A 300 ms-debounced press sets `sosActive`, starts a 15 s clear timer and emits urgent telemetry; a second press cancels. It is a local prototype alert, not an emergency service.

9. **How does the fall candidate work?** Free fall is `< 0.6 g`; an impact `> 2.5 g` between 60 and 600 ms later sets `fallActive` for 6 s and emits urgent telemetry. It is a heuristic, not clinically validated.

10. **How does hot-plug recovery work?** The 1 s report path probes installed devices; removal clears presence/value/raw. `updateSensorsHotplug()` retries initialization every 3 s, and TMP117/MAX have specific re-init paths.

11. **What does `active_max` mean and why is it 3?** It is the fixed denominator of installed measurement sensors: MAX30102, MPU and TMP117. OLED is a display; SHT40 and MLX90614 are absent and never counted.

12. **Why is the OLED text abbreviated?** The SSD1306 is 128×64 with a limited built-in font. Compact `TMP:`/`Wrist:` text fits; exact clinical labels and the full fever message are

implemented in the dashboard/JSON. Physical rendering is unverified.

13. **What happens if the TMP117 read is stuck at 0xFFFF?** The firmware rejects it as stuck_bus_raw_pattern, sets sensor_error, clears the value and does not add an offset. A raw diagnostic must confirm the hardware cause.

14. **Why does outputTelemetry() call refreshSensorStatuses() internally rather than trusting the caller?** Because urgent/fall paths can call it outside the normal one-second cadence. Refreshing there prevents a stale value from being published as valid during an emergency packet.

I.3 Five deeper follow-up questions

1. **"You use a cooperative loop. What happens if an I2C call hangs?"** The code sets a 25 ms Wire timeout, but it is not a hard real-time guarantee. A stuck bus or library call can still delay sampling and reporting. A production design would need watchdog/bus-recovery and task isolation.

2. **"Why is one packet per second enough for a patient monitor?"** It is a local prototype for trends, not waveform or arrhythmia monitoring. MPU updates happen faster internally, but the published state is reduced to the latest value. The HOD should hear that limitation explicitly.

3. **"Why not simply add a wrist-to-core offset?"** A fixed offset is not clinically valid; skin temperature is affected by ambient conditions, blood flow, contact pressure and activity. The repository intentionally prohibits it.

4. **"How do you know the status and value cannot disagree?"** The firmware uses shared validity predicates, and the bridge independently rejects contradictions (valid without value/sensor, non-valid with non-null value, raw mismatch, unverified wrist with value). Tests cover those paths.

5. **"What would you change before calling this a medical device?"** Validate every sensor against references, define contact and uncertainty, add clinical/regulatory review, security/privacy controls, fail-safe alerting and long-duration reliability testing. The current build is not that device.

I.4 Two cross-module questions

1. **"Your firmware emits a valid actual TMP117 reading while wrist contact is unverified. What must the dashboard show?"** The actual value as **TMP117 Sensor Temperature**

(°C); the clinical **Wrist Skin Temperature (°C)** card must show no numeric value, status `unverified`, with raw/timestamp hidden. Core remains unavailable and fever remains `not_available` with the exact message.

2. **"How does the bridge know that a packet is safe to store?"** `_validate_packet()` checks the schema, status/value consistency, ranges, sensor presence, timestamps, actual/wrist evidence, core/fever policy, and optional-sensor contradictions. Only then does `_accept_packet()` call `TelemetryStore.record()`; storage repeats the wrist/raw requirements in `measurements()`.

I.5 Three improvements

1. **Add an explicit firmware-level state-machine test harness for timing and recovery.** The current host tests cover protocol math and backend behavior, but not compile-and-run firmware scheduling. A hardware-in-the-loop test could verify stale transitions, hot-plug and urgent paths.

2. **Add bounded I2C recovery/watchdog behavior and observable bus-error counters per device.** The 25 ms timeout is useful but not a complete recovery strategy; production reliability needs bus reset/reinit policy.

3. **Replace OLED shorthand with a layout that is physically verified for legibility and clearly distinguishes actual sensor temperature from wrist status.** Keep the exact dashboard/JSON labels; validate contrast, font size and overflow on the real display.

I.6 Ten key facts

1. Firmware identity is `rpm-2.1.0`, schema `2`, serial `115200`, prefix `[JSON]`.
2. Installed measurement sensors are MAX30102, MPU and TMP117; `active_max` is 3.
3. OLED/MAX/TMP117 share `Wire` on GPIO 8/9 at 50 kHz; MPU uses `Wire1` on GPIO 17/18 at 400 kHz.
4. Normal telemetry is once per second; SOS/fall can emit immediately.
5. `null` is the only representation for unavailable/stale/invalid values.
6. Actual TMP117 temperature and clinical wrist temperature are separate fields.
7. Clinical wrist contact is always `unverified` in the current firmware.
8. Core is always null/`not_available`; fever is unavailable with the exact message.

9. SOS is active LOW on GPIO 5 with 300 ms debounce and 15 s auto-clear.

10. Fresh checks: 62 Python + 10 Node tests pass; both PlatformIO builds succeed; physical validation is missing.

I.7 Five terms

Term	Explanation
Validity state	The status that explains whether a measurement can be used (`valid`, `warming_up`, `low_quality`, `stale`, `sensor_error`, `not_available`).
Fail-closed	Prefer unavailable/null over a plausible number when evidence is insufficient.
Hot-plug	Detecting removal/reconnection at runtime and retrying initialization without a reboot.
Actual TMP117 temperature	The sensor-package reading; diagnostic telemetry, not wrist or core temperature.
Schema 2	The strict telemetry object version consumed by `telemetry_bridge.py`.

I.8 Three source files to open first

1. `src/main.cpp` — `setup()`, `loop()`, `outputTelemetry()`, `updateOLED()`, sensor init/update functions.

2. `src/tmp117_protocol.h` — the TMP117 constants, conversion, stuck/range helpers and compile-time checks.

3. `platformio.ini` + `boards/esp32-s3-devkitc-1-n16r8.json` — build target, disabled optional modules and board memory definition.

I.9 Three things never to claim without evidence

1. **"The TMP117 is accurate / the `-0.01 °C` issue is solved."** No reference measurement or captured raw diagnostic proves accuracy or the cause.

2. **"The SOS/fall/OLED behavior works on the device."** Source/build exists; assembled hardware tests were not performed.

3. **"The firmware provides wrist or core temperature."** The actual TMP117 value is not

clinical wrist/core, and the wrist/core fields remain unavailable.

I.10 Responsibility checklist

- ☐ I can trace `setup()` → `loop()` → `outputTelemetry()` without notes.
- ☐ I can explain every installed sensor and bus/address.
- ☐ I can explain why `active_max` is 3 and why SHT40/MLX are not counted.
- ☐ I can explain the status model and where staleness is applied.
- ☐ I can explain the exact actual-TMP117 versus clinical-wrist separation.
- ☐ I can explain why core is null and fever is `not_available`.
- ☐ I can explain SOS and fall paths and their limitations.
- ☐ I can explain the build flags and board manifest.
- ☐ I can show the fresh test/build results without calling them hardware validation.
- ☐ I can state the three "never claim" boundaries without prompting.
- ☐ I can answer two cross-module questions about bridge/dashboard behavior.
- ☐ I can name the next hardware verification steps.

J. Honest boundaries

J.1 What this member can defend

- The firmware's structure, scheduling and data contract.
- The status/null policy and its rationale.
- How the actual TMP117 field is kept separate from the clinical wrist field.
- How SOS/fall packets are triggered and why urgent output refreshes statuses.
- The build configuration, installed inventory and evidence classifications.

J.2 What this member must not claim

- Personal authorship of every file or prior repair.

- That the physical device, sensors or OLED were validated in this pass.
- That TMP117 readings are accurate, calibrated or clinically meaningful.
- That MAX30102 HR/SpO2 or MPU motion/fall results are clinically valid.
- That the device can diagnose fever or estimate core temperature.
- That SOS/fall constitutes emergency response.
- That SHT40/MLX90614 are installed or working.
- That the system uses a trained AI model or LLM.

J.3 Safe closing sentence

"The firmware integration is implemented and builds, and its host-level contract is tested. What remains is physical verification of the sensors, display, alerts and timing; until that happens I can defend the software design, not clinical or hardware accuracy."